

Software Engineering

Author

Software Engineering for Students

4th Edition, *by* DOUGLAS BELL

Presented by

Dr. Ghada Hamissa

Lecturer in Electrical Engineering Dept.
(Computer & Control System)

Faculty of Engineering



Chapter 1

Software Overview

Program and Software

- Set of instruction-> *program*.
- Set of program-> *software*.

This software is a collection of computer programs, procedure, rules and associative documentation and data.

Program is generally used the developer of a specific program to make a particular software.

Difference between program and software

program

- 1) Small in size.
- 2) Authors himself is user-soul.
- 3) Single developer.
- 4) Adopt development.
- 5) Lack proper interface.
- 6) Large proper documentation.

software

- 1) Large in size.
- 2) Large number.
- 3) Team developer.
- 4) Systematic development.
- 5) Well define interface.
- 6) Well documented.

Characteristics of software

- ✓ 1) Software is developed or engineer.
- ✓ 2) Most of software is custom build rather than assemble from existing component.
- ✓ 3) Computer program and associated documentation.
- ✓ 4) Easy to modified.
- ✓ 5) Easy to reproduce.
- ✓ 6) Software product may be developed for a particular customer or for the general market.

Application of software

- 1) System software.
- 2) Application software.
- 3) Engineering/scientific software.
- 4) Embedded software.
- 5) Product line software.
- 6) Web application software.
- 7) Artificial intelligence software (AI).

Software Overview

- **Software** is more than just a program code. A program is an executable code, which serves some computational purpose. Software is considered to be collection of executable programming code, associated libraries and documentations. Software, when made for a specific requirement is called **software product**.
- **Engineering** is all about developing products, using well-defined, scientific principles and methods.



What is the SWE?

- **Software engineering** is an engineering branch associated with development of software product using well-defined scientific principles, methods and procedures.
- The outcome of **software engineering** is an efficient and reliable software product.

Definitions

- **IEEE defines software engineering as**

(1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

(2) The study of approaches as in the above statement.

Fritz Bauer, a German computer scientist, defines software engineering as:

- **“Software engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and work efficiently on real machines.”**

Why software engineering ?

- 1) In the late 1960's hardware price were falling but software price rising.
- 2) Many software projects failed.
- 3) Large software project required large development loams.
- 4) Many software project late and over budget.
- 5) ^{تواطؤ} Complicity of software project is increased.
- 6) Demand for new software on the market.

Why study software engineering?

- 1) Higher productivity.
- 2) To acquire skills to develop large programs.
- 3) Ability to solve complex programming problems.
- 4) Learn techniques of specification design.
- 5) Better quality programmers.

Software Evolution

- The process of developing a software product using software engineering principles and methods is referred to as **Software Evolution (development)**.
- This includes the initial development of software and its maintenance and updates, till desired software product is developed, which satisfies the expected requirements.

Software Evolution Laws

Lehman has given laws for software evolution. He divided the software into three different categories:

1. **Static-type (S-type) -**
2. **Practical-type (P-type) –**
3. **Embedded-type (E-type) -**

Software Evolution Laws Con.

1. Static-type (S-type) –

- This is a software, which works strictly according to defined specifications and solutions.
- The solution and the method to achieve it, both are immediately understood before coding.
- The s-type software is least subjected to changes hence this is the simplest of all.
- For example, calculator program for mathematical computation.

Software Evolution Laws Con.

2. Practical-type (P-type) –

- This is a software with a collection of procedures. This is defined by exactly what procedures can do.
- In this software, the specifications can be described but the solution is not obviously instant.
- For example, gaming software.

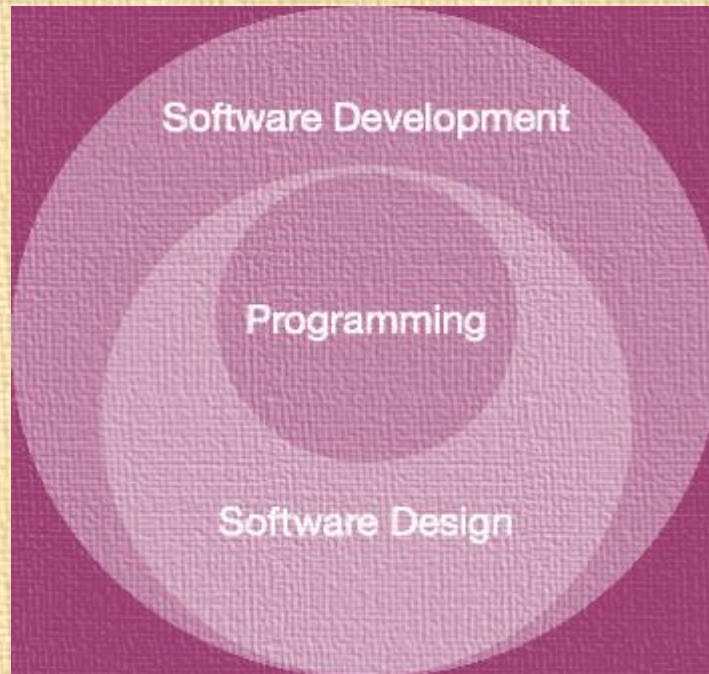
Software Evolution Laws Con.

3. Embedded-type (E-type) –

- This software works closely as the requirement of real-world environment.
- This software has a high degree of evolution as there are various changes in laws, taxes etc. in the real world situations.
- For example, Online trading software.

Software Paradigms

- **Software paradigms** refer to the methods and steps, which are taken while designing the software.



Characteristics of good software

1) Operational

- This tells us how well the software works in operations. It can be measured on:
 - ☐ **Budget**
 - ☐ **Usability**
 - ☐ **Efficiency**
 - ☐ **Correctness**
 - ☐ **Functionality**
 - ☐ **Dependability**
 - ☐ **Security**
 - ☐ **Safety**

Characteristics of good software

2) Transitional

- This aspect is important when the software is moved from one platform to another:
 - ☐ **Portability**
 - ☐ **Interoperability**
 - ☐ **Reusability**
 - ☐ **Adaptability**

Characteristics of good software

3) Maintenance

- This aspect briefs about how well the software has the capabilities to maintain itself in the ever-changing environment:
 - ☐ **Modularity**
 - ☐ **Maintainability**
 - ☐ **Flexibility**
 - ☐ **Scalability**

Finally

- In short, *Software engineering* is a branch of computer science, which uses well-defined engineering concepts required to produce efficient, durable, scalable, in-budget, and on-time software products.